

Relatório de Laboratório

Laboratório de Experimentação de Software - Lab01 - Repositórios populares + Setup do Kanban

Relatório final da análise de 1.000 repositórios populares do GitHub, com acompanhamento do processo no GitHub Projects.

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 - Repositórios populares + Setup do Kanban
Grupo (trio)	Ana Luiza Santos · Bruna Bernardes · Walter Louback
Link do repositório / GitHub Projects	bruna-bernardes/Lab_Experimentacao_Software https://github.com/users/bruna-bernardes/projects/2/views/1
Data de entrega	27/08/2026

1. Introdução

Repositórios com muitas estrelas no GitHub são frequentemente tratados como referências de engenharia de software. Entretanto, a popularidade, isoladamente, não explica características como maturidade, contribuição externa, frequência de releases, atividade de manutenção ou capacidade de encerrar issues. O Lab01 investiga essas propriedades nos 1.000 repositórios mais populares do GitHub, utilizando coleta automatizada pela API GraphQL e análise quantitativa.

As Questões de Pesquisa (RQs) do enunciado e as hipóteses informais formuladas antes da análise foram:

- RQ01 - Sistemas populares são maduros/antigos? Hipótese: a popularidade tende a se acumular ao longo de vários anos.
- RQ02 - Sistemas populares recebem muita contribuição externa? Hipótese: maior visibilidade tende a atrair mais pull requests.
- RQ03 - Sistemas populares lançam releases com frequência? Hipótese: projetos populares mantêm um processo ativo de entrega.
- RQ04 - Sistemas populares são atualizados com frequência? Hipótese: projetos populares apresentam atualização recente.
- RQ05 - Sistemas populares são escritos nas linguagens mais populares? Hipótese: a maior parte utiliza linguagens consolidadas no TIOBE Index.
- RQ06 - Sistemas populares possuem alto percentual de issues fechadas? Hipótese: projetos populares mantêm triagem e manutenção ativas.
- RQ07 - Linguagens populares x RQ02/RQ03/RQ04. Hipótese: comunidades maiores apresentariam mais PRs aceitas, mais releases e atualizações mais frequentes.

Como contribuição de inovação (30% da nota), o grupo acrescentou análises estatísticas não exigidas pelo enunciado: comparação de `closed_ratio` entre linguagens populares e demais (Mann-Whitney U),

correlação entre estrelas e `closed_ratio`, e correlações complementares entre estrelas x releases e estrelas x tempo desde a última atualização.

2. Contexto

O estudo foi desenvolvido ao longo das Sprints Lab01S01, Lab01S02 e Lab01S03. O objeto de estudo é uma amostra fixa de 1.000 repositórios ordenados por número de estrelas no GitHub. A coleta consolidada foi realizada em agosto de 2026 e armazenada em `data/repositorios_1000.csv`, permitindo que os scripts de análise reutilizem exatamente o mesmo conjunto de dados.

Para RQ05 e RQ07, a referência escolhida para classificar linguagens populares foi o TIOBE Index. A mesma lista foi mantida em todas as análises que dependem dessa classificação para evitar mudança de critério entre as sprints. O GitHub Projects (v2) foi utilizado como ferramenta de processo para organizar Issues, responsáveis, limite de WIP e snapshots do estado do Kanban.

A coleta evoluiu de 100 para 1.000 repositórios na Sprint 2. A paginação GraphQL usa `hasNextPage` e `endCursor`; durante os testes, páginas de 25 repositórios apresentaram recorrência de erros 502/504. A redução para 10 repositórios por página, combinada com retentativas automáticas e checkpoint, tornou a execução estável até completar os 1.000 registros.

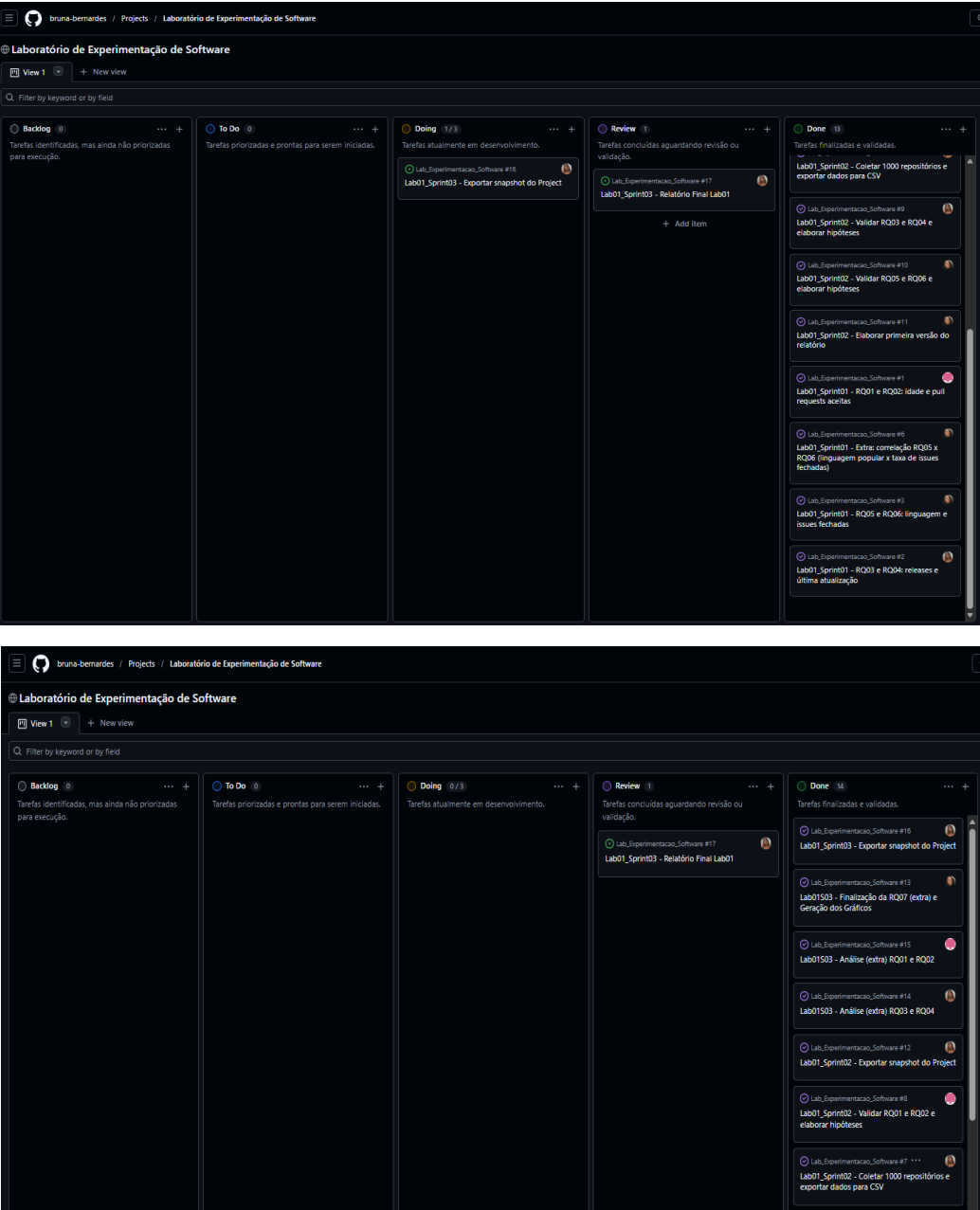
3. Metodologia

3.1 Principais Desafios

- Instabilidade temporária da API GraphQL (erros 502/503/504), mitigada com retentativas automáticas e backoff progressivo.
- Paginação de grande volume: a configuração inicial de 25 repositórios por página falhava com frequência; a execução com 10 por página apresentou maior estabilidade.
- Necessidade de retomar a coleta sem perder progresso: foi implementado checkpoint persistente com repositórios já coletados e cursor da próxima página.
- Distribuições assimétricas e presença de outliers, especialmente em pull requests e releases, exigindo uso de medianas, faixas e testes não paramétricos.
- Repositórios sem linguagem primária: 87 registros foram mantidos como categoria sem linguagem, em vez de serem descartados.

3.2 Tomadas de Decisão

- Utilizar o CSV consolidado como fonte única das análises, evitando chamadas redundantes à API e garantindo reprodutibilidade entre scripts.
- Adotar o TIOBE Index como referência única para popularidade de linguagens.
- Aplicar IQR para identificação de possíveis outliers nas validações das RQs.
- Utilizar Mann-Whitney U para comparação entre grupos quando não se assume normalidade e Spearman para relações monotônicas entre variáveis numéricas.
- Manter limite de WIP igual a 3 na coluna Doing, permitindo no máximo uma atividade simultânea por integrante do trio.



Quadro do GitHub Projects ao final da Sprint 3, apresentado em duas capturas para visualizar todas as Issues.

3.3 Etapas

A divisão abaixo reflete as Issues mantidas no repositório e no GitHub Projects ao longo das três sprints, bem como seus respectivos responsáveis.

Sprint	Entregas	Responsável(is)	Issues (nº)
Lab01S01	RQ01/RQ02: coleta e validação inicial	Walter	#1
Lab01S01	RQ03/RQ04: coleta e validação inicial	Bruna	#2
Lab01S01	RQ05/RQ06: coleta e validação inicial	Ana	#3
Lab01S01	Extra: RQ05 x RQ06 (issues fechadas)	Ana	#6
Lab01S02	Paginação, 1.000 repositórios e CSV	Bruna	#7
Lab01S02	Validação RQ01/RQ02	Walter	#8
Lab01S02	Validação RQ03/RQ04 e elaboração de hipóteses	Bruna	#9
Lab01S02	Validação RQ05/RQ06 e elaboração de hipóteses	Ana	#10
Lab01S02	Primeira versão do relatório	Ana	#11
Lab01S02	Exportação do snapshot da Sprint 2	Bruna	#12

Lab01S03	Finalização da RQ07 e geração dos gráficos	Ana	#13
Lab01S03	Análise extra RQ03/RQ04	Bruna	#14
Lab01S03	Análise extra RQ01/RQ02	Walter	#15
Lab01S03	Exportação do snapshot final do Project	Bruna	#16
Lab01S03	Relatório Final do Lab01	Bruna	#17

Configuração do processo e snapshot final

- Fluxo do board: Backlog -> To Do -> Doing -> Review -> Done.
- Limite de WIP em Doing: 3 itens, permitindo no máximo uma tarefa ativa por integrante do trio.
- As tarefas foram rastreadas por Issues e associadas aos responsáveis reais no GitHub Projects.
- Os snapshots Lab01S01.csv, Lab01S02.csv e Lab01S03.csv foram gerados e versionados no repositório, registrando o estado do GitHub Projects ao final de cada sprint.

3.4 Ferramentas

- Python 3 - scripts de coleta, validação, análise estatística e geração de gráficos.
- GitHub GraphQL API - coleta automatizada de repositórios, estrelas, datas, PRs, releases, linguagens e issues.
- requests - chamadas HTTP ao endpoint GraphQL.
- csv/json - persistência dos dados, checkpoints e resumos de análise.
- matplotlib - visualizações das RQs e análises complementares.
- scipy.stats - Mann-Whitney U e correlação de Spearman.
- GitHub Projects (v2) - Kanban, atribuição de responsáveis, limite de WIP e snapshots por sprint.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data de análise - createdAt	Anos	GraphQL + Python
RQ02	Pull requests aceitas	Total de PRs com estado MERGED	Contagem	GraphQL + Python
RQ03	Total de releases	releases.totalCount	Contagem	GraphQL + Python
RQ04	Tempo desde última atualização	Data de análise - updatedAt	Dias	GraphQL + Python
RQ05	Linguagem primária	primaryLanguage.name comparada ao TIOBE	Categórica	GraphQL + TIOBE
RQ06	Razão de issues fechadas	closedIssues / totalIssues; 0 se totalIssues = 0	Proporção [0,1]	GraphQL + Python
RQ07	RQ02/RQ03/RQ04 por popularidade da linguagem	Medianas comparadas entre linguagens TIOBE e demais	Conforme RQ02-04	Python + scipy.stats
Extra	RQ05 x RQ06 + correlações	Mann-Whitney U e Spearman	p-valor; rho	scipy.stats

3.6 Inovações Propostas pelo Grupo (30% da nota)

A principal inovação estatística cruza RQ05 (linguagem primária) e RQ06 (razão de issues fechadas). Os repositórios foram segmentados entre linguagens populares no TIOBE e demais linguagens. O teste de Mann-Whitney U verifica se as distribuições de closed_ratio diferem entre os grupos; adicionalmente, Spearman foi usado para investigar se estrelas se relacionam com a taxa de fechamento de issues.

Uma segunda frente complementar aprofundou RQ03 e RQ04: foi testada a correlação entre estrelas e total de releases e entre estrelas e dias desde a última atualização. Essas análises não substituem as RQs originais; elas verificam se a popularidade medida em estrelas explica diretamente as métricas observadas.

Os resultados dessas inovações são apresentados na seção 4 e retomados na discussão e conclusão, conforme exigido pelo template.

4. Resultados

4.1 Coleta de Dados

A coleta final atingiu exatamente 1.000 repositórios e foi exportada para `data/repositorios_1000.csv`. Todos os registros possuem as métricas numéricas necessárias às RQ01-RQ04 e RQ06; em RQ05, 87 repositórios não possuíam linguagem primária informada e foram mantidos explicitamente como categoria sem linguagem. As validações foram versionadas em scripts separados para RQ01/RQ02, RQ03/RQ04 e RQ05/RQ06.

A amostra apresenta forte assimetria em algumas métricas. Em RQ02, por exemplo, a mediana é 770 PRs aceitas, enquanto a média é 4.243,3 e o máximo chega a 103.403. Em RQ03, 277 repositórios possuem zero releases, enquanto outros chegam a 1.000. Por isso, a interpretação privilegia medianas, distribuições por faixa e testes não paramétricos.

4.2 Visualização Gráfica

RQ01 - Sistemas populares são maduros/antigos?

A idade média foi de 7,67 anos e a mediana de 7,76 anos. A maior concentração está entre 9 e 15 anos (436 repositórios), enquanto 49 possuem mais de 15 anos.

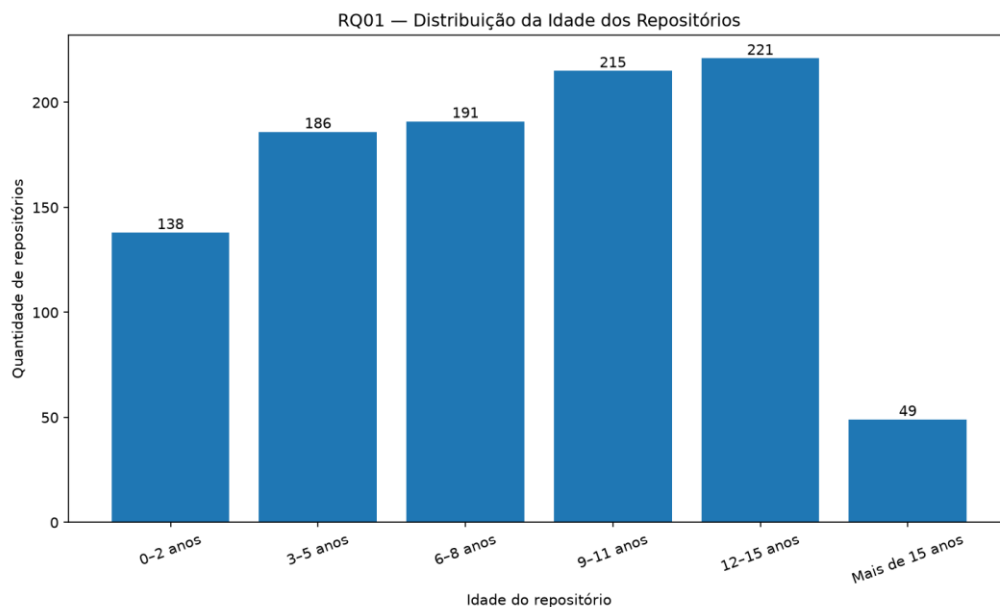


Figura 1 - Distribuição da idade dos 1.000 repositórios.

RQ02 - Sistemas populares recebem muita contribuição externa?

A mediana foi de 770 pull requests aceitas. A maior parte da amostra está entre 101 e 10.000 PRs aceitas: 368 repositórios na faixa 101-1.000 e 351 na faixa 1.001-10.000.

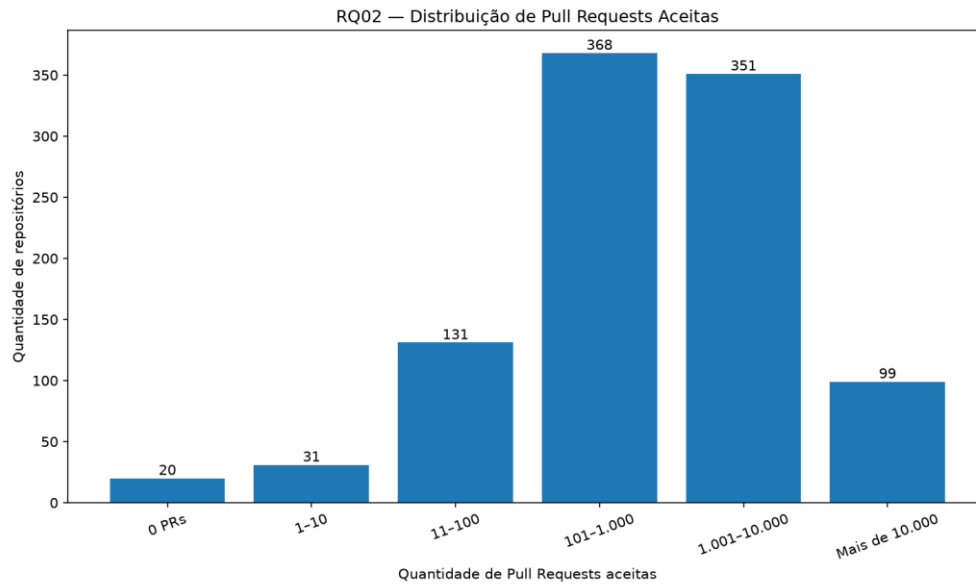


Figura 2 - Distribuição da quantidade de pull requests aceitas.

RQ03 - Sistemas populares lançam releases com frequência?

A mediana foi de 40 releases e a média de 127,95. A distribuição é heterogênea: 277 repositórios não possuem releases publicados, enquanto 348 possuem mais de 100 releases.

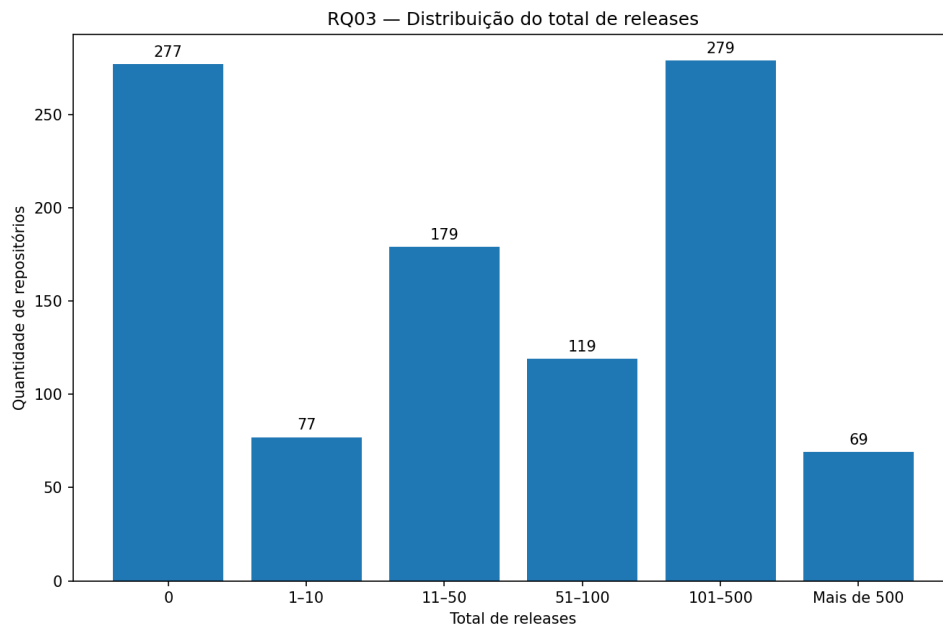


Figura 3 - Distribuição do total de releases.

RQ04 - Sistemas populares são atualizados com frequência?

Na execução final da análise, os 1.000 repositórios apresentaram atualização muito recente: o intervalo observado foi de 6 a 8 dias desde updatedAt. Pela segmentação usada no gráfico, 999 estavam na faixa de 1-7 dias e apenas 1 na faixa de 8-30 dias.

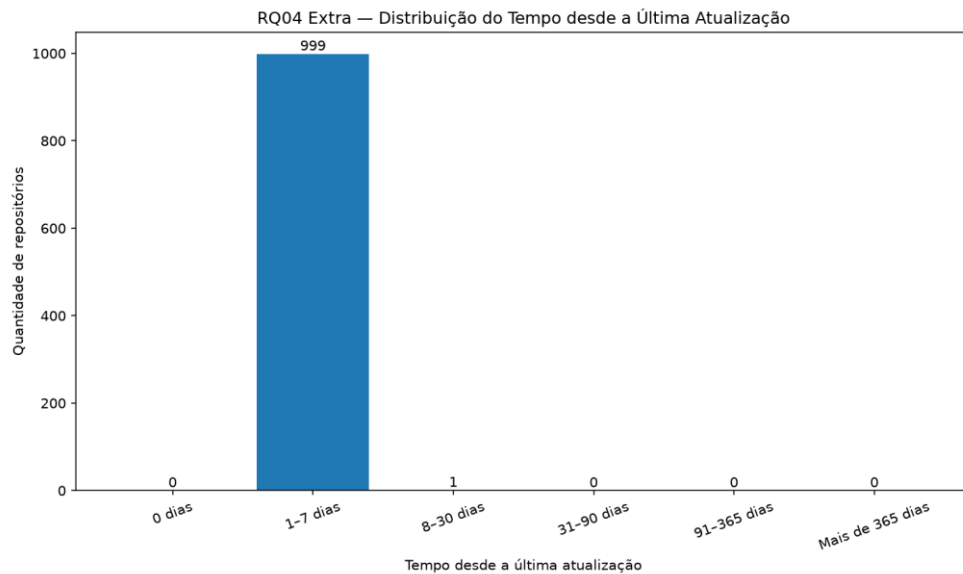


Figura 4 - Distribuição do tempo desde a última atualização.

RQ05 - Sistemas populares são escritos nas linguagens mais populares?

Das 1.000 observações, 793 foram classificadas em linguagens da lista TIOBE utilizada pelo grupo, 120 em outras linguagens e 87 sem linguagem primária informada. Python, TypeScript e JavaScript aparecem como as três categorias mais frequentes.

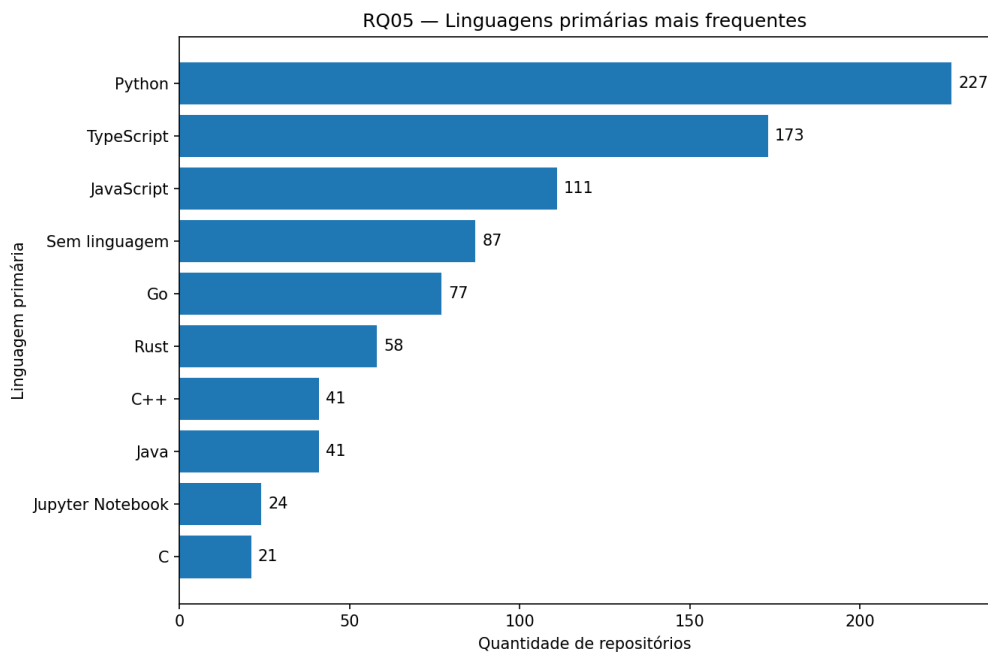


Figura 5 - Linguagens primárias mais frequentes na amostra.

RQ06 - Sistemas populares possuem alto percentual de issues fechadas?

A mediana de `closed_ratio` foi 0,8645 e a média 0,7679. Ao todo, 615 repositórios apresentaram razão de issues fechadas igual ou superior a 0,80, enquanto 43 apresentaram razão igual a zero.

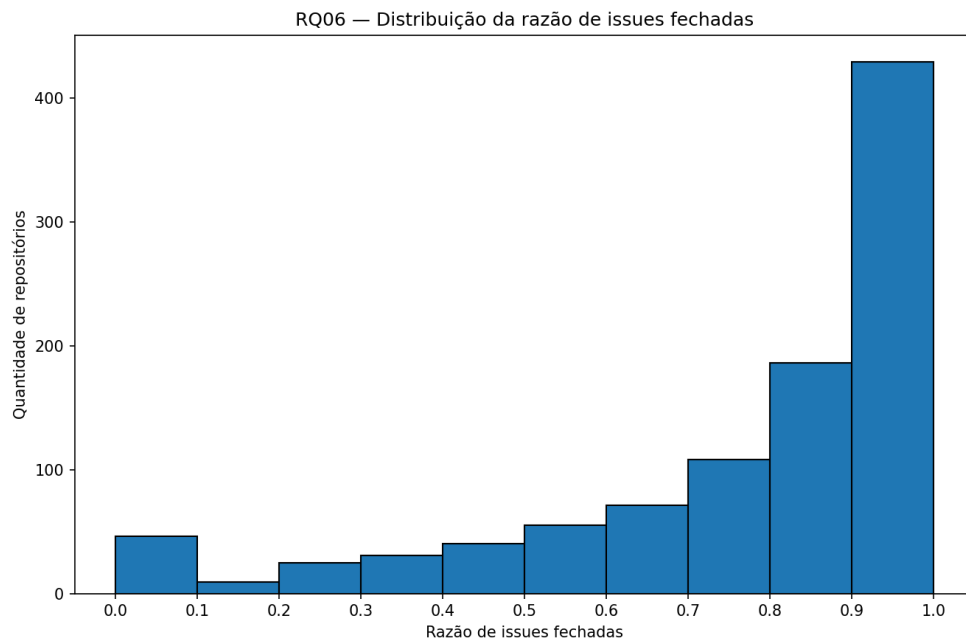


Figura 6 - Distribuição da razão de issues fechadas.

RQ07 - Linguagens populares x contribuição, releases e atualização

Na comparação entre linguagens populares e demais, as medianas foram 1.074 vs. 315 PRs aceitas ($p < 0,001$) e 67 vs. 1 release ($p < 0,001$). Para dias desde a última atualização, as medianas registradas na execução da RQ07 foram 5 vs. 5 dias, sem diferença significativa ($p = 0,757422$). Como essa métrica depende da data de execução, esses valores diferem ligeiramente da análise final da RQ04, executada posteriormente.

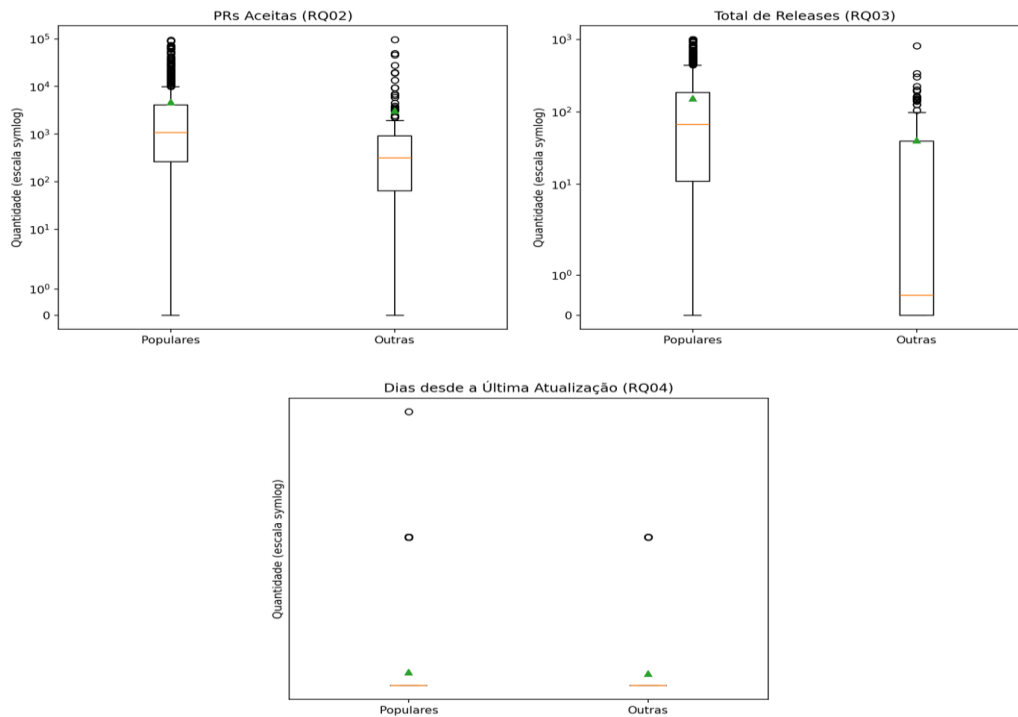


Figura 7 - Comparações da RQ07 entre linguagens populares e demais.

Resultados das inovações (30%)

Na inovação RQ05 x RQ06, a mediana de `closed_ratio` foi 0,8808 para linguagens populares e 0,7873 para as demais. O teste de Mann-Whitney U resultou em $p = 0,000455$, indicando diferença estatisticamente significativa. Entretanto, a correlação entre estrelas e `closed_ratio` foi praticamente nula ($\rho = 0,006$; $p = 0,849302$).

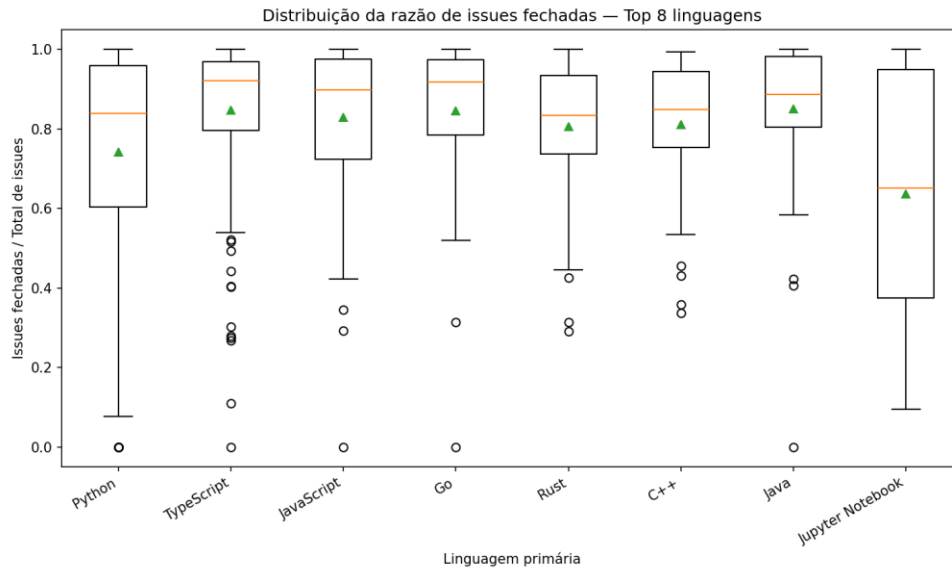


Figura 8 - Boxplot de `closed_ratio` por linguagem, usado na análise complementar RQ05 x RQ06.

Na análise complementar de RQ03/RQ04, estrelas x releases apresentou $\rho = -0,022$ ($p = 0,487141$) e estrelas x dias desde a última atualização apresentou $\rho = -0,0542$ ($p = 0,086977$). Em ambos os casos não foi identificada correlação estatisticamente significativa.

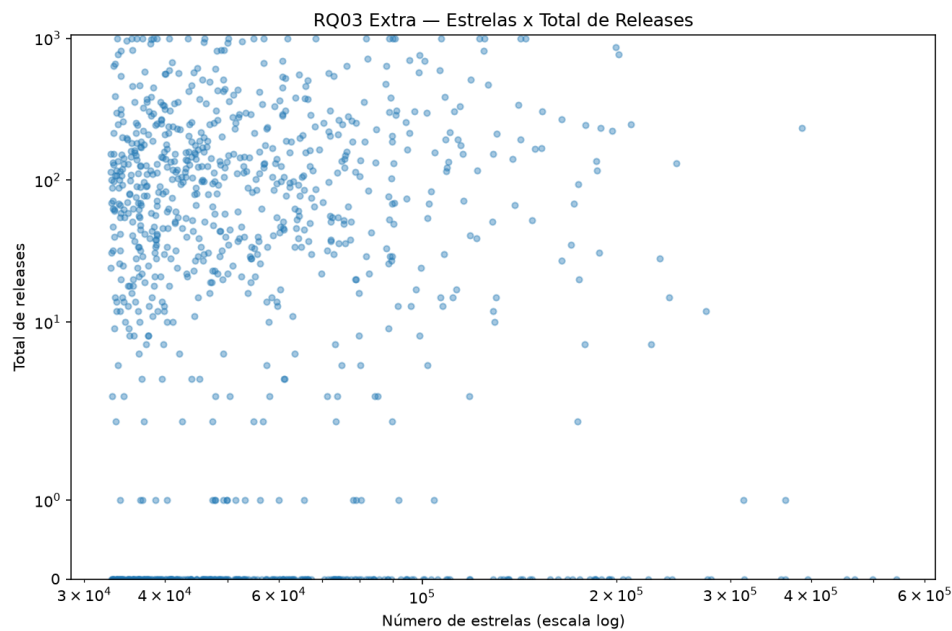


Figura 9 - Estrelas x total de releases na análise complementar da RQ03.

4.3 Discussão

RQ01: a hipótese de maturidade foi confirmada de forma geral. A mediana próxima de oito anos e a concentração entre 9 e 15 anos mostram forte presença de projetos maduros, embora também existam repositórios recentes extremamente populares.

RQ02: a hipótese de alto volume de contribuição externa foi confirmada. A mediana de 770 PRs e a predominância de projetos com centenas ou milhares de PRs aceitas demonstram intensa participação externa; a distância entre média e mediana revela a presença de grandes outliers.

RQ03: a hipótese de releases frequentes foi parcialmente confirmada. Muitos projetos apresentam dezenas ou centenas de releases, mas 277 não utilizam releases do GitHub. Além disso, a quantidade de estrelas não apresentou correlação significativa com total de releases.

RQ04: a hipótese de atualização frequente foi confirmada descritivamente, pois todos os repositórios estavam atualizados há no máximo oito dias na análise final. Contudo, a baixa variabilidade impediu observar relação significativa entre estrelas e tempo de atualização.

RQ05: a hipótese foi apoiada: 793 dos 1.000 repositórios utilizam linguagens incluídas na lista TIOBE adotada. A existência de 87 projetos sem linguagem primária e 120 em outras linguagens mostra, porém, que popularidade do repositório não depende exclusivamente de uma linguagem dominante.

RQ06: a hipótese de alta proporção de issues fechadas foi apoiada pela mediana de 0,8645. A inovação mostrou ainda diferença significativa entre linguagens populares e demais, mas estrelas não explicaram a taxa de fechamento.

RQ07: a hipótese foi parcialmente confirmada. Linguagens populares apresentaram mais PRs aceitas e mais releases, com diferenças estatisticamente significativas; por outro lado, não houve diferença significativa no tempo desde a última atualização.

A principal ameaça à validade está na natureza observacional do estudo: estrelas representam popularidade, mas não causalidade. O uso de `updatedAt` também torna RQ04 dependente da data de execução. Além disso, a amostra contém projetos de naturezas distintas (frameworks, listas curadas, materiais educacionais, sistemas e bibliotecas), o que afeta a interpretação de releases, issues e PRs.

5. Conclusão

O Lab01 concluiu a mineração e análise de 1.000 repositórios populares do GitHub com coleta automatizada por GraphQL, paginação robusta, checkpoint, exportação em CSV e validações específicas por RQ. Os resultados mostram que projetos populares tendem a ser maduros, recebem grande volume de contribuições e apresentam atualização recente; também há forte presença de linguagens populares e alta proporção de issues fechadas.

As RQs relacionadas a releases e linguagem exigem interpretação mais cuidadosa: nem todo projeto usa releases do GitHub da mesma forma e a popularidade do repositório não é explicada apenas pela linguagem. A RQ07 mostrou diferenças claras em PRs e releases entre linguagens populares e demais, mas não em frequência de atualização.

As inovações estatísticas acrescentaram uma conclusão importante: número de estrelas, isoladamente, não apresentou correlação significativa com releases, tempo desde a última atualização ou taxa de

fechamento de issues. Portanto, estrelas são um indicador de visibilidade, mas não devem ser tratadas como medida direta de qualidade ou manutenção.

6. Referências

ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.

TIOBE Software. TIOBE Index. Disponível em: <https://www.tiobe.com/tiobe-index/>.

GitHub. GraphQL API Documentation. Disponível em: <https://docs.github.com/en/graphql>.

GitHub. About Projects. Disponível em: <https://docs.github.com/en/issues/planning-and-tracking-with-projects>.

YOUTUBE. Vídeo disponibilizado como referência da disciplina. Disponível em: <https://www.youtube.com/shorts/YwnaeO95AN8>.